

Name: M Venkata Sai
Reg No:192425223

MLA02_CO2-AT3

MLA 0201-FUNDAMENTAL OF MACHINE LEARNING
Debugging & Optimisation Task.

Q1. Linear Regression – High Prediction Error

(i) Problem Identification – 20%

Possible causes of high prediction error are:

- **Multicollinearity:** Independent features are highly correlated.
- **Outliers:** Extremely high/low house prices affect the regression line.
- **Poor feature selection:** Important features such as area, bedrooms and age may be missing.
- Features may be on different scales.

(ii) Debugging – 25%

```
import pandas as pd
```

```
import numpy as np
```

```
data = {  
    "Area": [1000, 1500, 800, 1200, 2000, 2500, 3000, 700],  
    "Bedrooms": [2, 3, 2, 3, 4, 5, 5, 1],  
    "Age": [10, 5, 15, 8, 3, 2, 1, 30],  
    "Price": [50, 75, 40, 60, 90, 120, 140, 35]  
}
```

```
df = pd.DataFrame(data)
```

```
# Check correlation
```

```
print("Correlation:")
```

```
print(df.corr())
```

```
# Detect outliers using IQR
```

```
Q1 = df["Price"].quantile(0.25)
```

```
Q3 = df["Price"].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower = Q1 - 1.5 * IQR
```

```
upper = Q3 + 1.5 * IQR
```

```
outliers = df[(df["Price"] < lower) | (df["Price"] > upper)]
```

```
print("\nOutliers:")
```

```
print(outliers)
```

(iii) Optimization – 20%

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import mean_squared_error, r2_score
```

```
# Remove outliers
```

```
df = df[(df["Price"] >= lower) & (df["Price"] <= upper)]
```

```
X = df[["Area", "Bedrooms", "Age"]]
```

```
y = df["Price"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```

X, y, test_size=0.2, random_state=42
)

# Feature scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train optimized model
model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2 Score:", r2_score(y_test, y_pred))

```

(iv) Analysis – 20%

- Correlation analysis detects multicollinearity.
- IQR detects abnormal price values.
- Standardization puts features on a common scale.
- RMSE measures prediction error.
- R^2 measures how well the model explains house prices.

Conclusion: Removing outliers, scaling features and selecting relevant features improves regression performance.

(v) Code Quality – 15%

The code is divided into **data preparation, debugging, optimization and evaluation**, making it easy to understand and modify.

Q2. Email Classification – Spam Classified as Non-Spam

(i) Problem Identification

Spam classified as non-spam is a **False Negative (FN)**.

Possible causes:

- Imbalanced dataset.
- Poor decision boundary.
- High classification threshold.
- Insufficient text features.

(ii) Debugging

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report
```

```
emails = [
    "Win free money now",
    "Congratulations you won a prize",
    "Claim free reward",
    "Meeting at 10 AM",
    "Send the project report",
    "Class starts at 9 AM",
    "Free lottery winner",
    "Project meeting tomorrow"
]
```

```
# 1 = spam, 0 = normal
```

```
y = [1, 1, 1, 0, 0, 0, 1, 0]
```

```
vectorizer = TfidfVectorizer()
```

```
X = vectorizer.fit_transform(emails)
```

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = LogisticRegression(class_weight="balanced")
model.fit(X_train, y_train)

prob = model.predict_proba(X_test)[:, 1]

# Lower threshold to catch more spam
y_pred = (prob >= 0.40).astype(int)

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print(classification_report(y_test, y_pred))

```

(iii) Optimization

- TF-IDF improves text representation.
- `class_weight="balanced"` handles class imbalance.
- Threshold 0.40 makes the model more sensitive to spam.
- Retraining with balanced data improves minority-class detection.

(iv) Analysis

The most important metric is **Recall** because we want to reduce spam emails incorrectly classified as normal.

(v) Code Quality

The code clearly separates text preprocessing, training, threshold optimization and evaluation.

Conclusion: Balanced training and threshold adjustment reduce false-negative spam predictions.

Q3. Naïve Bayes – Zero Probability

(i) Problem Identification

Naïve Bayes can produce a probability of zero when a feature value does not occur in the training data.

Because probabilities are multiplied, one zero probability can make the complete class probability zero.

(ii) Debugging and Optimization

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
messages = [
    "free money offer",
    "win free prize",
    "meeting tomorrow",
    "project discussion",
    "free lottery winner",
    "class meeting today"
]
```

```
# 1 = spam, 0 = normal
```

```
y = [1, 1, 0, 0, 1, 0]
```

```
vectorizer = CountVectorizer()
```

```
X = vectorizer.fit_transform(messages)
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.3, random_state=42
```

)

```
# Laplace smoothing
```

```
model = MultinomialNB(alpha=1.0)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("Predictions:", y_pred)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

(iii) Analysis

alpha=1.0 applies **Laplace smoothing**.

Adding 1 prevents zero probabilities.

Conclusion: Laplace smoothing allows Naïve Bayes to handle unseen features and improves classification reliability.

Q4. Discriminant Function – Poor Class Separation

(i) Problem Identification

Possible causes:

- Overlapping classes.
- Incorrect weights.
- Different feature scales.
- Irrelevant features.

(ii) Debugging

```
import numpy as np
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
```

```
from sklearn.model_selection import train_test_split
```

```

from sklearn.metrics import accuracy_score

X = np.array([
    [2, 3], [3, 4], [4, 5], [5, 6],
    [6, 7], [7, 8], [8, 9], [9, 10]
])

y = np.array([0, 0, 0, 0, 1, 1, 1, 1])

# Normalize
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.25, random_state=42
)

model = LinearDiscriminantAnalysis()
model.fit(X_train, y_train)

prediction = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, prediction))
print("Feature coefficients:", model.coef_)

```

(iii) Optimization

- Standardize features.
- Examine discriminant coefficients.
- Remove irrelevant features.
- Adjust/retrain model parameters.

(iv) Analysis

The discriminant function is:

The learned coefficients indicate the contribution of each feature.

Conclusion: Normalization and improved feature selection can create better class separation.

Q5. Logistic Regression – Overfitting

(i) Problem Identification

Signs:

- Training accuracy is very high.
- Testing accuracy is much lower.
- Training loss decreases while validation loss increases.

(ii) Debugging

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

```
X, y = make_classification(
    n_samples=500,
    n_features=20,
    n_informative=5,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Regularized model
```

```

model = LogisticRegression(
    penalty="l2",
    C=0.1,
    max_iter=1000
)

model.fit(X_train, y_train)

train_pred = model.predict(X_train)
test_pred = model.predict(X_test)

print("Training Accuracy:",
      accuracy_score(y_train, train_pred))

print("Testing Accuracy:",
      accuracy_score(y_test, test_pred))

```

(iii) Optimization

L2 regularization reduces excessively large coefficients.

```
penalty="l2"
```

```
C=0.1
```

Smaller C means stronger regularization.

Cross-validation can also be used to select the best value of C.

(iv) Analysis

Compare training and testing accuracy. If the gap becomes smaller after regularization, generalization has improved.

Conclusion: Regularization controls model complexity and reduces overfitting.

Q6. Linear Regression – Different Feature Scales

(i) Problem Identification

Advertising budget may be in thousands while the number of outlets may be in units.

This difference in scale can make optimization inefficient.

(ii) Debugging

```
import numpy as np
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.preprocessing import StandardScaler
```

```
X = np.array([
```

```
    [10, 5],
```

```
    [20, 10],
```

```
    [30, 15],
```

```
    [40, 20],
```

```
    [50, 25]
```

```
])
```

```
y = np.array([20, 35, 50, 65, 80])
```

```
# Before scaling
```

```
model1 = LinearRegression()
```

```
model1.fit(X, y)
```

```
print("Before Scaling:")
```

```
print(model1.coef_)
```

```
# Standardization
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
model2 = LinearRegression()
```

```
model2.fit(X_scaled, y)
```

```
print("\nAfter Scaling:")
```

```
print(model2.coef_)
```

(iii) Optimization

Standardization:

makes features comparable.

(iv) Analysis

After scaling, gradient-based optimization can work more efficiently and coefficient magnitudes become easier to interpret.

Conclusion: Standardization improves numerical stability and optimization.

Q7. Fraud Detection – Class Imbalance

(i) Problem Identification

If 95% of transactions are normal and only 5% are fraudulent, a model can achieve high accuracy while detecting very few fraud cases.

(ii) Debugging

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
```

```
X, y = make_classification(
    n_samples=1000,
    n_features=10,
    weights=[0.95, 0.05],
    random_state=42
)
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y, test_size=0.3, random_state=42  
)
```

```
model = LogisticRegression(  
    class_weight="balanced",  
    max_iter=1000  
)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print(classification_report(y_test, y_pred))
```

(iii) Optimization

```
class_weight="balanced"
```

automatically gives greater importance to the minority class.

(iv) Analysis

Use:

For fraud detection, **recall is particularly important**.

Conclusion: Class weighting improves detection of fraudulent transactions.

Q8. Naïve Bayes – Correlated Features

(i) Problem Identification

Naïve Bayes assumes feature independence. Highly correlated features violate this assumption.

(ii) Debugging

```
import numpy as np
```

```
import pandas as pd
```

```
from sklearn.datasets import load_breast_cancer
```

```
data = load_breast_cancer()
```

```
X = pd.DataFrame(  
    data.data,  
    columns=data.feature_names  
)
```

```
# Correlation analysis
```

```
corr = X.corr().abs()
```

```
upper = corr.where(  
    np.triu(  
        np.ones(corr.shape),  
        k=1  
    ).astype(bool)  
)
```

```
high_corr = [  
    column for column in upper.columns  
    if any(upper[column] > 0.90)  
]
```

```
print("Highly correlated features:")
```

```
print(high_corr)
```

(iii) Optimization

Remove highly correlated features:

```
X_reduced = X.drop(columns=high_corr)
```

```
print("Original features:", X.shape[1])  
print("Reduced features:", X_reduced.shape[1])
```

Then train Naïve Bayes again.

(iv) Analysis

Features with correlation above 0.90 contain highly similar information. Removing redundant features makes the data more appropriate for Naïve Bayes.

Conclusion: Feature selection reduces redundancy and may improve classification accuracy.

Q9. Logistic Regression – Overlapping Feature Distributions

(i) Problem Identification

The two classes have overlapping feature distributions, so a simple linear decision boundary cannot separate them effectively.

(ii) Debugging

```
from sklearn.datasets import make_classification  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import PolynomialFeatures  
from sklearn.pipeline import Pipeline  
from sklearn.preprocessing import StandardScaler  
from sklearn.linear_model import LogisticRegression  
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
X, y = make_classification(  
    n_samples=500,  
    n_features=2,  
    n_redundant=0,  
    n_informative=2,  
    class_sep=0.7,  
    random_state=42  
)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
model = Pipeline([
    ("polynomial", PolynomialFeatures(degree=2)),
    ("scaler", StandardScaler()),
    ("logistic", LogisticRegression())
])
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
print("Accuracy:",
      accuracy_score(y_test, y_pred))
```

(iii) Optimization

Polynomial features add terms such as:

This allows logistic regression to learn a nonlinear decision boundary.

Conclusion: Feature transformation can improve classification when the original feature space has overlapping classes.

Q10. Discriminant Analysis – Underfitting

(i) Problem Identification

Underfitting occurs when the model is too simple.

Signs:

- Low training accuracy.
- Low testing accuracy.
- High bias.
- Important patterns are not captured.

(ii) Debugging

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.discriminant_analysis import (
    LinearDiscriminantAnalysis,
    QuadraticDiscriminantAnalysis
)
from sklearn.metrics import accuracy_score
```

```
X, y = make_classification(
    n_samples=500,
    n_features=5,
    n_informative=4,
    random_state=42
)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
# Simple model
```

```
lda = LinearDiscriminantAnalysis()
lda.fit(X_train, y_train)
```

```
lda_pred = lda.predict(X_test)
```

```
print("LDA Accuracy:",  
      accuracy_score(y_test, lda_pred))  
  
# More flexible model  
qda = QuadraticDiscriminantAnalysis()  
qda.fit(X_train, y_train)  
  
qda_pred = qda.predict(X_test)  
  
print("QDA Accuracy:",  
      accuracy_score(y_test, qda_pred))
```

(iii) Optimization

QDA is more flexible than LDA because it can create a **quadratic/nonlinear decision boundary**.

Other improvements:

- Add relevant features.
- Use nonlinear transformations.
- Increase model complexity.
- Remove irrelevant features.

(iv) Analysis

Compare LDA and QDA accuracy. If QDA performs better, the original linear model may have been too simple for the dataset.

Conclusion: Increasing model flexibility can reduce underfitting and improve predictive performance.